Объектно-ориентированное программирование

4 семестр
Лекция 4.

Содержание лекции:

- Многопоточность
- Класс Thread
- Потоки с параметрами
- ThreadPool
- Синхронизация потоков

Преподаватель курса: Клюкин Даниил Анатольевич, ст. преподаватель каф. ПМиИТ

Многопоточность

Поток — это основная единица, которой операционная система выделяет время процессора.

При помощи многопоточности можно выделить в приложении несколько потоков, которые будут выполнять различные задачи одновременно.

Например, обработку данных и отображение результатов можно выделить в разные потоки, чтобы на время расчета программа не «зависала».

Многопоточность может быть реализована 3 способами:

1. Thread
2. ThreadPool
3. Task

Основной функционал для использования потоков в приложении сосредоточен в пространстве имен **System.Threading**. В нем определен класс, представляющий отдельный поток - класс **Thread**.

Плюсы:

✅ Полный контроль над потоком (запуск, приостановка, приоритеты).

✅ Подходит для долгосрочных задач.

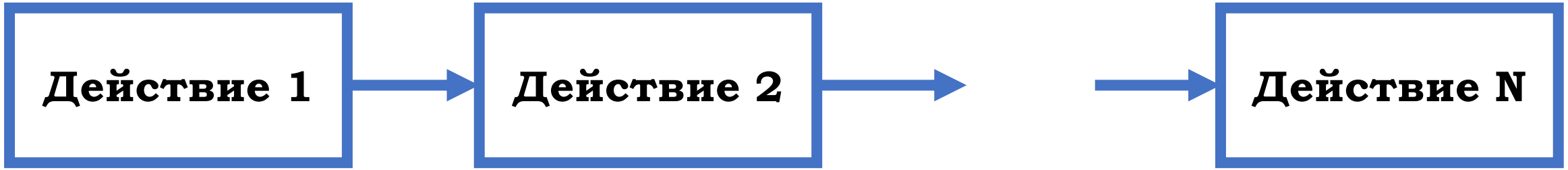
✅ Возможность ручного управления состоянием потока.

Минусы:

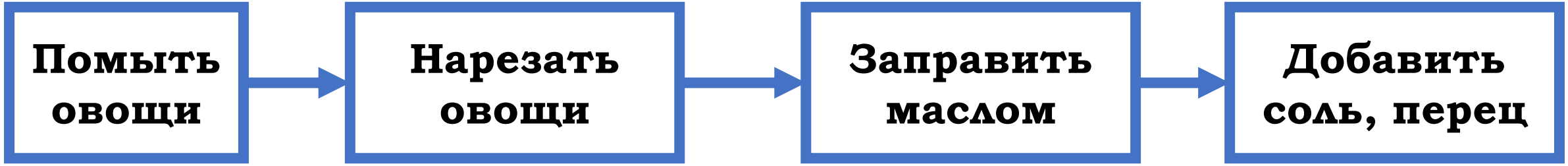
❌ Создание потока – дорогая операция (каждый поток требует ресурсов).

❌ Большое количество потоков может привести к перегрузке системы.

Ключевое отличие однопоточного режима от многопоточного: один процесс выполняется одним исполнителем.



Однопоточный режим выполнения на примере:



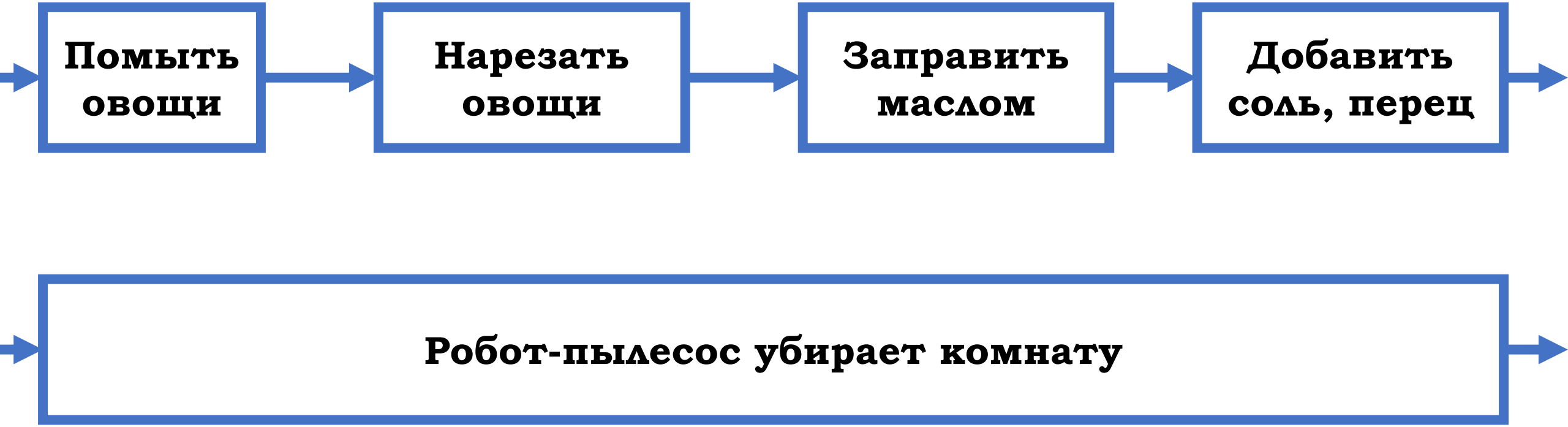
```
Print();   // 1 вызов метода
Console.WriteLine("Некоторые действия в методе Main"); // 4

void Print()
{
    Thread.Sleep(3000);     // 2 имитация продолжительной работы
    Console.WriteLine("Hello World"); // 3 Вывод в консоль
}
```

```
Hello World
Некоторые действия в методе Main
```

В многопоточном режиме в рамках одного процесса решаются несколько задач разными потоками.

Многопоточный режим на примере домашних дел:



Основные свойства класса Thread:

статическое свойство **CurrentThread** позволяет получить текущий поток;

IsAlive: указывает, работает ли поток в текущий момент;

Name: содержит имя потока;

ManagedThreadId: возвращает числовой идентификатор текущего потока;

Priority: хранит приоритет потока - значение перечисления **ThreadPriority**:

- **Lowest**;
- **BelowNormal**;
- **Normal** (по умолчанию);
- **AboveNormal**;
- **Highest**.

В программе на C# есть как минимум один поток - **главный поток**, в котором выполняется метод Main.

```
Имя потока:
Имя потока: Метод Main
Запущен ли поток: True
Id потока: 1
Приоритет потока: Normal
Статус потока: Running
```

```
using System.Threading;
// получаем текущий поток
Thread currentThread = Thread.CurrentThread;
//получаем имя потока
Console.WriteLine($"Имя потока: {currentThread.Name}");
currentThread.Name = "Метод Main";
Console.WriteLine($"Имя потока: {currentThread.Name}");
Console.WriteLine($"Запущен ли поток: {currentThread.IsAlive}");
Console.WriteLine($"Id потока: {currentThread.ManagedThreadId}");
Console.WriteLine($"Приоритет потока: {currentThread.Priority}");
Console.WriteLine($"Статус потока: {currentThread.ThreadState}");
```

Также класс **Thread** определяет ряд методов для управления потоком.

Основные из них:

- Статический метод **Sleep** останавливает поток на определенное количество миллисекунд;
- Метод **Interrupt** прерывает поток, который находится в состоянии WaitSleepJoin;
- Метод **Join** блокирует выполнение вызвавшего его потока до тех пор, пока не завершится поток, для которого был вызван данный метод;
- Метод **Start** запускает поток.

Например, применим метод **Sleep** для задания задержки выполнения приложения:

```
using System.Threading;

for (int i = 0; i < 10; i++)
{
    Thread.Sleep(500);// задержка выполнения на 500 миллисекунд
    Console.WriteLine(i);
}
```

Примеры создания потоков:

```
Thread myThread1 = new Thread(Print);
Thread myThread2 = new Thread(new ThreadStart(Print));
Thread myThread3 = new Thread(() => Console.WriteLine("Hello
Threads"));

void Print()
{
    Thread.Sleep(2000);
    Console.WriteLine($"Hello Threads
      {Thread.GetCurrentProcessorId()}");
}
```

```
Hello Threads
Hello Threads 3
Hello Threads 5
```

Для запуска нового потока применяется метод **Start** класса **Thread**:

```
myThread1.Start();  // запускаем поток myThread1
myThread2.Start();  // запускаем поток myThread2
myThread3.Start();  // запускаем поток myThread3
```

Запустим потоки в цикле:

```
using System.Threading;
// создаем новый поток
Thread myThread = new Thread(Print);
myThread.Start(); // запускаем поток myThread
for (int i = 0; i < 5; i++)
{// действия, выполняемые в главном потоке
    Console.WriteLine($"Главный поток: {i}");
    Thread.Sleep(400);
}
void Print()
{// действия, выполняемые во втором потоке
    for (int i = 0; i < 5; i++)
    {
        Console.WriteLine($"Второй поток: {i}");
        Thread.Sleep(400);
    }
}
```

```
Главный поток: 0
Второй поток: 0
Главный поток: 1
Второй поток: 1
Второй поток: 2
Главный поток: 2
Второй поток: 3
Главный поток: 3
Второй поток: 4
Главный поток: 4
```

Thread.Join – блокирует вызывающий поток до завершения потока, представленного этим экземпляром.

```
static void DoWork()
{
    Console.WriteLine(
        $"Поток {Thread.CurrentThread.ManagedThreadId} выполняет
работу.");
}

Thread thread = new Thread(DoWork);
thread.Start();

Console.WriteLine(
    $"Главный поток {Thread.CurrentThread.ManagedThreadId}
продолжает работу.");
thread.Join(); // Ожидание завершения потока
```

Потоки с параметрами

Пример 1:

```
using System.Threading;
// создаем новые потоки
Thread myThread1 = new Thread(new
ParameterizedThreadStart(Print));
Thread myThread2 = new Thread(Print);
Thread myThread3 = new Thread(message =>
Console.WriteLine(message));
// запускаем потоки
myThread1.Start("Hello");
myThread2.Start("Привет");
myThread3.Start("Salut");
void Print(object? message)
{
    if (message is string msgText)
        Console.WriteLine(msgText);
}
```

Пример 2:

```
using System.Threading;

int number = 4;
// создаем новый поток
Thread myThread = new Thread(Print);
myThread.Start(number);    // n * n = 16


// действия, выполняемые во втором потокке
void Print(object? obj)
{
    // здесь мы ожидаем получить число
    if (obj is int n)
    {
        Console.WriteLine($"n * n = {n * n}");
    }
}
```

Пример 3:

```
class Person
{
    public Person(string name, int age)
    {
        Name = name;
        Age = age;
    }

    public string Name { get; set; }
    public int Age { get; set; }
}
```

```
using System.Threading;

Person tom = new Person("Tom", 37);
// создаем новый поток
Thread myThread = new Thread(Print);
myThread.Start(tom);

void Print(object? obj)
{
    // здесь мы ожидаем получить объект Person
    if (obj is Person person)
    {
        Console.WriteLine($"Name = {person.Name}");
        Console.WriteLine($"Age = {person.Age}");
    }
}
```

2. ThreadPool – это пул управляемых потоков, предоставляемый CLR для выполнения коротких задач.

Плюсы:

✅ Эффективное использование ресурсов (потоки создаются заранее и переиспользуются).

✅ Автоматическое управление количеством потоков.

✅ Подходит для множества коротких задач.

Минусы:

❌ Невозможно контролировать поток напрямую (например, приоритет).

❌ Не подходит для долгих операций (может забить пул).

Основные методы

ThreadPool.QueueUserWorkItem() – добавляет задачу в очередь пула.

```
ThreadPool.QueueUserWorkItem(DoWork, "Hello, ThreadPool!");
```

ThreadPool.SetMinThreads() / ThreadPool.SetMaxThreads() – настройка количества потоков.

```
ThreadPool.SetMinThreads(10, 10); // Минимум 10 рабочих и I/O потоков
ThreadPool.SetMaxThreads(100, 100); // Максимум 100 потоков
```

☞ По умолчанию (зависит от ядер CPU):

Min ≈ количество ядер

Max ≈ 32 767 (но реально ограничено системой)

Пример использования пула потоков:

```
static void DoWork(object? state)
{
    Console.WriteLine($"Поток из пула
{Thread.CurrentThread.ManagedThreadId} выполняет задачу.");
}

ThreadPool.QueueUserWorkItem(DoWork);
Console.WriteLine(
   $"Главный поток {Thread.CurrentThread.ManagedThreadId}
продолжает работу.");

Thread.Sleep(1000); // Даем время потоку из пула завершиться
```

Синхронизация потоков. lock

Все потоки, принадлежащие одному процессу, разделяют некоторые общие ресурсы (адресное пространство, открытые файлы).

Когда два или более потоков имеют доступ к одному разделенному ресурсу, они нуждаются в обеспечении того, что ресурс будет использован только одним потоком одновременно. Процесс, с помощью которого это достигается, называется **синхронизацией**.

Проблема: Гонка данных (Race Condition)

Допустим, у нас есть общий счетчик, который увеличивается из нескольких потоков. Без синхронизации возможна потеря данных.

```
int counter = 0;
void IncrementCounter()
{
    for (int i = 0; i < 10000; i++)
    {
        counter++; // Небезопасно!
    }
}
Thread thread1 = new Thread(IncrementCounter);
Thread thread2 = new Thread(IncrementCounter);

thread1.Start();
thread2.Start();

thread1.Join();
thread2.Join();
Console.WriteLine($"Итоговое значение: {counter}");
// Может быть меньше 20000 из-за гонки данных!
```

```
Итоговое значение: 12043
```

Решение: Создание **критической секции** – участка кода, который выполняется только одним потоком в момент времени.

lock — это механизм синхронизации в C#, который обеспечивает безопасный доступ к общим ресурсам из нескольких потоков.

```
object locker = new(); // Объект-заглушка
int counter = 0;

void IncrementCounter()
{
    for (int i = 0; i < 10000; i++)
    {
        lock (locker)
            counter++;
    }
}
```

Пример Deadlock (взаимная блокировка)

```
void Thread1()
{
    lock (lock1)
    {
        Thread.Sleep(100);
        lock (lock2) { }
    } // Будет ждать, пока Thread2 освободит lock2
}
void Thread2()
{
    lock (lock2)
    {
        Thread.Sleep(100);
        lock (lock1) { }
    } // Будет ждать, пока Thread1 освободит lock1
}
```

Напишем код для многопоточного расчета числа Пи.

```
var iterations = (long)1e10;

object locker = new();
var pi = CalculatePiMultithread(iterations);
Console.WriteLine(pi);
void CalculatePiThread(long from, long to, ref double pi)
{
    var subSum = 0.0;
    for (long i = from; i < to; i++)
    {
        var sign = i % 2 == 0 ? 1 : -1;
        subSum += sign / (2 * i + 1.0);
    }
    Console.WriteLine($"{Thread.CurrentThread.Name} Готово!");
    lock (locker)
        pi += 4 * subSum;
}
```

```
double CalculatePiMultithread(long n)
{
    var pi = 0.0;
    var threadsCount = Environment.ProcessorCount;
    var threads = new Thread[threadsCount];
    var iterationsOnThread = (double)n / threadsCount;
    for (int i = 0; i < threadsCount; i++)
    {
        var from = (long)(i * iterationsOnThread);
        var to = (long)((i + 1) * iterationsOnThread);
        Thread myThread=new(()=>CalculatePiThread(from,to,ref pi));
        myThread.Name = $"Поток {i}";
        threads[i] = myThread;
        myThread.Start();
    }
    //Ждем пока потоки досчитают
    foreach (var thread in threads) thread.Join();
    return pi;
}
```

Результат многопоточного расчета числа Пи.

```
Поток 6 Готово!
Поток 9 Готово!
Поток 3 Готово!
Поток 2 Готово!
Поток 5 Готово!
Поток 1 Готово!
Поток 7 Готово!
Поток 10 Готово!
Поток 8 Готово!
Поток 4 Готово!
Поток 0 Готово!
Поток 11 Готово!
3,14159265348822
```

Monitor

Monitor — это механизм синхронизации в .NET, обеспечивающий взаимное исключение (mutual exclusion) при работе с многопоточными приложениями.

- Основные возможности:
- Блокировка критических секций (аналогично lock).
- Возможность ожидания с таймаутом.
- Гибкое управление потоками через сигналы (Pulse, PulseAll, Wait).
- Отличие от lock:
- lock — это синтаксический сахар над Monitor.Enter и Monitor.Exit.
- Monitor предоставляет больше контроля (например, таймауты, условные блокировки).

Метод	**Описание**
Monitor.Enter(obj)	Захватывает блокировку объекта (если возможно).
Monitor.Exit(obj)	Освобождает блокировку объекта.
Monitor.TryEnter(obj, timeout)	Пытается захватить блокировку с таймаутом (избегает deadlock).
Monitor.Wait(obj)	Временно освобождает блокировку и ждет сигнала (Pulse/PulseAll).
Monitor.Pulse(obj)	Пробуждает **один** поток из ожидающих (Wait).
Monitor.PulseAll(obj)	Пробуждает **все** потоки из ожидающих (Wait).

Пример использования Monitor

```
using System.Diagnostics.Metrics;
object lockObj = new object();
int counter = 0;
void SafeIncrement()
{
    Monitor.Enter(lockObj);
    try
    {
        // Критическая секция
        counter++;
    }
    finally
    {
        Monitor.Exit(lockObj); // Важно освободить блокировку!
    }
}
```

Пример использования TryEnter (избегаем deadlock)

```
object lockObj = new object();

if (Monitor.TryEnter(lockObj, 1000)) // Ждем 1 секунду
{
    try
    {
        // Критическая секция
    }
    finally
    {
        Monitor.Exit(lockObj);
    }
}
else
    Console.WriteLine("Не удалось захватить блокировку!");
```

✅ Ситуации, где Monitor предпочтительнее lock:

- Нужен таймаут при захвате блокировки (TryEnter).
- Требуется условная синхронизация (Wait/Pulse).
- Важен более тонкий контроль над блокировками.

❌ Когда не стоит использовать:

- Для простых сценариев (лучше lock).
- Для межпроцессной синхронизации (нужен Mutex).

Mutex

Mutex (Mutual Exclusion) - это примитив синхронизации, который позволяет организовать взаимоисключающий доступ к общему ресурсу между потоками в рамках одного процесса или между разными процессами.

Особенности Mutex

✅ Преимущества:

- Поддержка межпроцессной синхронизации
- Возможность рекурсивных блокировок
- Поддержка таймаутов при ожидании
- Системное имя позволяет синхронизировать разные приложения

✕ Недостатки:

- Более медленный, чем lock или Monitor (работает на уровне ядра ОС)
- Требует явного освобождения (в отличие от lock)
- Необходимость обработки исключений (например, при аварийном завершении потока)

Метод	**Описание**
WaitOne()	Блокирует мьютекс (если он свободен)
ReleaseMutex()	Освобождает мьютекс
WaitOne(таймаут)	Попытка захвата с ожиданием в течение указанного времени
Close()/Dispose()	Освобождение системных ресурсов

Базовый пример (внутрипроцессная синхронизация)

```
Mutex mutex = new Mutex();
int counter = 0;
for (int i = 0; i < 5; i++)
    new Thread(Increment).Start();
void Increment()
{
    mutex.WaitOne(); // Захватываем мьютекс
    try
    {
        for (int i = 0; i < 10000; i++)
            counter++;
        Console.WriteLine($"Counter = {counter}");
    }
    finally
        mutex.ReleaseMutex(); // Освобождаем мьютекс
}
```

Межпроцессная синхронизация

```
// Создаем именованный мьютекс (видимый другим процессам)
using (var mutex = new Mutex(false, "Global\\MyAppMutex")) {
    // Пытаемся захватить мьютекс (ждем 5 секунд)
    if (!mutex.WaitOne(5000))
    {
        Console.WriteLine("Другая копия приложения уже
запущена!");
        return;
    }
    try {
        Console.WriteLine("Приложение работает...");
        Console.ReadLine();
    }
    finally {
        mutex.ReleaseMutex();
    }
}
```

Semaphore

Semaphore — это примитив синхронизации, который ограничивает количество потоков, одновременно имеющих доступ к ресурсу.

🔹 Основные характеристики:

- Поддерживает заданное максимальное число потоков в критической секции.
- Может быть именованным (для межпроцессной синхронизации).
- Использует счетчик, уменьшающийся при захвате (WaitOne) и увеличивающийся при освобождении (Release).

📌 Отличие от Mutex:

- Mutex разрешает доступ только одному потоку (как lock).
- Semaphore разрешает доступ N потокам одновременно.

Метод	**Описание**
WaitOne()	Захватывает семафор (уменьшает счетчик). Если счетчик = 0 — блокирует поток.
Release()	Освобождает семафор (увеличивает счетчик), разрешая доступ другому потоку.
WaitOne(таймаут)	Пытается захватить семафор с ожиданием (возвращает false, если не удалось).
Dispose()	Освобождает системные ресурсы (рекомендуется использовать using).

```
Semaphore semaphore = new Semaphore(3, 3); // Максимум 3 потока
for (int i = 1; i <= 10; i++)
{
    new Thread(DoWork).Start(i);
    void DoWork(object? id) {
        Console.WriteLine($"Поток {id} ждет доступа...");

        semaphore.WaitOne(); // Захватываем семафор
        try {
            Console.WriteLine($"Поток {id} вошел в крит. с.");
            Thread.Sleep(2000); // Имитация работы
        }
        finally {
            semaphore.Release(); // Освобождаем семафор
            Console.WriteLine($"Поток {id} вышел.");
        }
    }
}
```

Когда использовать Semaphore?

✅ Популярные сценарии:

- Ограничение подключений к БД/API.
- Управление доступом к общему файлу.
- Реализация пула потоков (например, ограничение параллельных задач).
- Для высоконагруженных приложений используйте SemaphoreSlim.

❌ Когда не стоит использовать:

- Если нужна эксклюзивная блокировка (лучше Mutex).
- Для простых сценариев (иногда хватает lock).